Name: BANDAM AKARSH
Roll.No: 2303A53018
batch-46

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week8 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:16.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*

1	**Lab 16 – Database Design and Queries: Schema Design and SQL Generation**
	Lab Objectives:
	• Understand schema design principles for relational databases.
	• Use AI-assisted tools to generate **SQL schema definitions**.
	• Write and refine **SQL queries** (CRUD + aggregate operations).
	• Explore how AI can optimize queries and ensure normalization.
	
	Task 1 – Hospital Management Database
	Task:
	Create schema and queries for a **Hospital Management System**.
	Instructions:
	• Tables: Doctors, Patients, Appointments.
	• Use AI to define constraints (unique IDs, valid dates).
	• Generate queries:
	  • List all appointments for a specific doctor.
	  • Retrieve patient history by patient ID.
	  • Count total patients treated by each doctor.
	Expected Output:
	• Normalized schema and SQL queries with joins.
	CODE:
	
	DROP TABLE IF EXISTS Appointments;
	DROP TABLE IF EXISTS Patients;
	DROP TABLE IF EXISTS Doctors;
	
	CREATE TABLE Doctors (
	  doctor_id INT PRIMARY KEY,
	  name VARCHAR(100),
	  specialization VARCHAR(100)
	);
	
	CREATE TABLE Patients (
	  patient_id INT PRIMARY KEY,
	  name VARCHAR(100),
	  age INT,
	  gender VARCHAR(10)
	);
	
	CREATE TABLE Appointments (
	  appointment_id INT PRIMARY KEY,
	  doctor_id INT,
	  patient_id INT,
	  appointment_date DATE,
	  FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
	  FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
	);
	
	INSERT INTO Doctors VALUES
	(1, 'Dr. Smith', 'Cardiology'),
		Week8 - Wednesday

```
(2, 'Dr. John', 'Neurology');

INSERT INTO Patients VALUES
(101, 'Alice', 25, 'Female'),
(102, 'Bob', 30, 'Male');

INSERT INTO Appointments VALUES
(1, 1, 101, '2025-08-01'),
(2, 1, 102, '2025-08-02'),
(3, 2, 101, '2025-08-03');

SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;


SELECT a.appointment_id, a.appointment_date, p.name AS patient_name
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;

SELECT a.appointment_id, a.appointment_date, d.name AS doctor_name
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 101;

SELECT d.name, COUNT(DISTINCT a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

Task 2 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an **E-commerce platform**.
Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
  - Retrieve all orders by a user.
  - Find the most purchased product.
  - Calculate total revenue in a given month.
- AI should also suggest **normalization improvements** and **query optimization**.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

CODE:

```
DROP TABLE IF EXISTS OrderDetails;
```

```sql
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

CREATE TABLE Users (
   user_id INT PRIMARY KEY,
   name VARCHAR(100),
   email VARCHAR(100) UNIQUE
);

CREATE TABLE Products (
   product_id INT PRIMARY KEY,
   name VARCHAR(100),
   price DECIMAL(10,2)
);

CREATE TABLE Orders (
   order_id INT PRIMARY KEY,
   user_id INT,
   order_date DATE,
   FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails (
   order_detail_id INT PRIMARY KEY,
   order_id INT,
   product_id INT,
   quantity INT,
   FOREIGN KEY (order_id) REFERENCES Orders(order_id),
   FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

INSERT INTO Users VALUES
(1, 'Alice', 'alice@email.com'),
(2, 'Bob', 'bob@email.com');

INSERT INTO Products VALUES
(1, 'Laptop', 50000),
(2, 'Phone', 20000);

INSERT INTO Orders VALUES
(1, 1, '2025-08-01'),
(2, 1, '2025-08-10'),
(3, 2, '2025-08-15');

INSERT INTO OrderDetails VALUES
(1, 1, 1, 1),
(2, 2, 2, 2),
(3, 3, 2, 1);
```

```
-- 1. Orders by user
SELECT * FROM Orders WHERE user_id = 1;

-- 2. Most purchased product
SELECT p.name, SUM(od.quantity) AS total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 1;

-- 3. Total revenue (August 2025)
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE MONTH(o.order_date) = 8 AND YEAR(o.order_date) = 2025;
```

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
  - Retrieve all books currently issued.
  - Find overdue books (loan date > 30 days).
  - Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

CODE:

```
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Members;
DROP TABLE IF EXISTS Books;

CREATE TABLE Books (
   book_id INT PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(100)
);

CREATE TABLE Members (
   member_id INT PRIMARY KEY,
   name VARCHAR(100)
);

CREATE TABLE Loans (
```

```
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

INSERT INTO Books VALUES
(1, 'DBMS', 'Author A'),
(2, 'AI Basics', 'Author B');

INSERT INTO Members VALUES
(1, 'John'),
(2, 'Alice');

INSERT INTO Loans VALUES
(1, 1, 1, '2025-07-01', NULL),
(2, 2, 2, '2025-06-01', NULL);


-- 1. Books currently issued
SELECT b.title, m.name
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;

-- 2. Overdue books (>30 days)
SELECT b.title, m.name
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE DATEDIFF(CURDATE(), l.loan_date) > 30
AND l.return_date IS NULL;

-- 3. Books per member
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name;
```

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

```
-- Original query
```

SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

```
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.

CODE:

```
SELECT * FROM Books
WHERE author_id IN (
   SELECT author_id FROM Authors WHERE country='UK'
);

SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Task 5: University Course Registration

Scenario:

SR University needs a **course registration system** for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
  - Each student can register for multiple courses.
  - Each course is taught by a faculty member.
- Write SQL queries to:
  - List all students enrolled in a specific course.
  - Find faculty members teaching more than 2 courses.
  - Retrieve courses with the highest number of registrations.

CODE:

```
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;
DROP TABLE IF EXISTS Students;

CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   name VARCHAR(100)
);
```

```
CREATE TABLE Faculty (
   faculty_id INT PRIMARY KEY,
   name VARCHAR(100)
);

CREATE TABLE Courses (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(100),
   faculty_id INT,
         FOREIGN KEY (faculty_id) REFERENCES
Faculty(faculty_id)
);

CREATE TABLE Registrations (
   registration_id INT PRIMARY KEY,
   student_id INT,
   course_id INT,
         FOREIGN KEY (student_id) REFERENCES
Students(student_id),
         FOREIGN KEY (course_id) REFERENCES
Courses(course_id)
);

INSERT INTO Students VALUES
(1, 'Alice'),
(2, 'Bob'),
(3, 'Charlie');

INSERT INTO Faculty VALUES
(1, 'Prof. A'),
(2, 'Prof. B');

INSERT INTO Courses VALUES
(101, 'DBMS', 1),
(102, 'AI', 1),
(103, 'Networks', 2);

INSERT INTO Registrations VALUES
(1, 1, 101),
(2, 2, 101),
(3, 3, 102),
(4, 1, 102),
(5, 2, 103);


-- 1. Students in a course
SELECT s.name
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
WHERE r.course_id = 101;
```

```
-- 2. Faculty teaching >2 courses
SELECT f.name, COUNT(c.course_id) AS total_courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.name
HAVING COUNT(c.course_id) > 2;

-- 3. Course with highest registrations
SELECT c.course_name, COUNT(r.student_id) AS
total_students
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_name
ORDER BY total_students DESC
LIMIT 1;
```